1. TITLE PAGE

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous Institution, Accredited by NAAC 'A' Grade, Approved by AICTE, New Delhi)

Department of Artificial Intelligence and Data Science

Arasur, Coimbatore – 641 407, Tamil Nadu, India

CASE STUDY

Empirical Investigation and Deep Learning Architecture for Voluntary Employee Turnover: High-Dimensional Diagnostic Visualization and Predictive Risk Modeling

Course Code & Title	U21ADP05 – Exploratory Data Analysis and Visualization
Assignment	Assignment II (40 Marks)
Academic Year & Semester	2026 – 2027 (ODD SEM) | III Year / Semester V
Batch	2024 – 2028
Faculty Evaluator	Department Course Evaluation Committee
Student Name	Raveena S
Roll Number	24AD087
Degree & Branch	B.Tech. – Artificial Intelligence and Data Science
Date of Submission	15 / 10 / 2026
Evaluation Rubrics	Total 40 Marks (CO3: 8 | CO4: 16 | CO5: 16 + Deep Learning Enrichment)
GitHub Repository	github.com/raveenasivaprakasam021/IBM-HR-Attrition-EDA-DL

2. Abstract

Voluntary employee attrition introduces substantial operational friction, intellectual capital depreciation, and severe financial replacement costs within enterprise ecosystems. This academic study presents an end-to-end exploratory data visualization and deep learning diagnostic framework using the canonical IBM HR Analytics Employee Attrition Dataset (N = 1,470 instances, 35 attributes). Rigorously implementing Ben Fry's 7 Stages of Data Visualization, we execute moment diagnostics, Tukey's interquartile range outlier fencing, non-parametric Mann-Whitney U testing, and categorical Chi-Square tests of independence. To advance beyond classical linear modeling, a 3-layer deep feed-forward Multi-Layer Perceptron (PyTorch MLP) with Batch Normalization and Dropout regularization was engineered, hyperparameter-tuned across three architectural variants, and benchmarked against regularized Logistic Regression. The deep architecture achieved 84.16% test accuracy, an ROC-AUC of 0.8284, and an out-of-sample sensitivity (recall) of 66.67%, outperforming the linear baseline by +22.23% in capturing true flight-risk personnel. Permutation feature importance identified mandatory overtime, job involvement, and promotion stagnation as primary turnover drivers.

(Word Count: ≈148 words – within the required 100–150 word specification.)

84.16%	0.8284
Test Accuracy	ROC-AUC
66.67%	**+22.23%**
Recall (24/36)	Recall vs baseline

3. Introduction & Objective

3.1 Industrial Background & Economic Motivation

Replacing a skilled departed worker consumes between 50% and 200% of that individual's annualized base salary [1]. These costs comprise recruitment agency commissions, talent acquisition interview hours, signing bonuses, and a 6-to-9 month productivity ramp-up latency.

3.2 Technical Objectives

- **Step 1 – Structural integrity & hygiene:** null/duplicate diagnostics; prune zero-variance fields.
- **Step 2 – Statistical distribution diagnostics:** skewness, kurtosis and anomaly detection via Tukey's IQR fences.
- **Step 3 – Hypothesis testing:** Mann-Whitney U (continuous) and Pearson Chi-Square (categorical).
- **Step 4 – Exploratory visual analytics:** Ben Fry's 7 Stages across 6 diverse figures.
- **Step 5 – Deep learning architecture:** 3-hidden-layer PyTorch MLP with BatchNorm, ReLU, Dropout and L2 weight decay, tuned across 3 configurations.
- **Step 6 & 7 – Interpretation & benchmarking:** permutation feature importance (5 repeats) and comparison with regularized Logistic Regression.

4. Dataset Description

The analysis uses the real-world IBM HR Analytics Employee Attrition dataset: 1,470 employee records and 35 fields covering demographics, job characteristics, compensation, tenure history and satisfaction surveys.

4.1 Feature Schema Dictionary

Table 1: Feature taxonomy of the IBM HR Attrition dataset

Category	Fields
Demographic	Age, Gender, MaritalStatus, Education, EducationField
Job profile	Department, JobRole, JobLevel, JobInvolvement, JobSatisfaction, BusinessTravel, OverTime, PerformanceRating
Compensation	MonthlyIncome, DailyRate, HourlyRate, MonthlyRate, PercentSalaryHike, StockOptionLevel
Tenure & career	TotalWorkingYears, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion, YearsWithCurrManager, NumCompaniesWorked, TrainingTimesLastYear
Well-being	EnvironmentSatisfaction, RelationshipSatisfaction, WorkLifeBalance, DistanceFromHome
Pruned	EmployeeCount, Over18, StandardHours, EmployeeNumber
Target	Attrition (Yes = departed, No = retained)

4.2 Descriptive Summary Statistics of Continuous Variables

Table 2: Descriptive statistics and moment diagnostics (N = 1,470)

Feature	Mean	Std	Min	Median	Max	Skew	Kurt
Age	36.92	9.14	18.0	36.0	60.0	+0.413	−0.407
DailyRate	802.49	403.51	102.0	802.0	1499.0	−0.003	−1.206
DistanceFromHome	9.19	8.11	1.0	7.0	29.0	+0.958	−0.225
MonthlyIncome	6,502.93	4,707.96	1,009	4,919	19,999	+1.368	+0.998
TotalWorkingYears	11.28	7.78	0.0	10.0	40.0	+1.116	+0.911
YearsAtCompany	7.01	6.13	0.0	5.0	40.0	+1.765	+3.924

5. EDA and Preprocessing

5.1 Preprocessing Pipeline Execution

1. **Missing values & duplicates:** zero missing values and zero duplicate rows, confirming sample independence.
2. **Zero-variance pruning:** EmployeeCount (1), Over18 ('Y'), StandardHours (80) and EmployeeNumber (ID) removed.
3. **Outlier detection (Tukey IQR):** see Table 3. MonthlyIncome outliers are Executive VPs/Directors and were retained.
4. **Encoding & scaling:** categorical features encoded to give D = 44 inputs; continuous features standardized ($\mu = 0$, $\sigma = 1$) fit on the training partition only.
5. **Stratified splitting:** 70/15/15 with random_state = 42 (Table 4).

Table 3: Tukey's IQR outlier fences ($F_L = Q1 - 1.5 \cdot IQR$, $F_U = Q3 + 1.5 \cdot IQR$)

Feature	Q1	Median	Q3	IQR	Upper	Outliers
MonthlyIncome	$2,911	$4,919	$8,379	$5,468	$16,581	114 (7.76%)
TotalWorkingYears	6.0	10.0	15.0	9.0	28.5	63 (4.29%)
YearsAtCompany	3.0	5.0	9.0	6.0	18.0	104 (7.07%)
Age	30.0	36.0	43.0	13.0	62.5	0 (0.00%)
DistanceFromHome	2.0	7.0	14.0	12.0	32.0	0 (0.00%)

Table 4: Stratified train / validation / test partitions

Partition	Share	Rows	Retained	Attrited
Training	70%	1,029	863	166 (16.13%)
Validation	15%	220	185	35 (15.91%)
Test	15%	221	185	36 (16.29%)
Total	100%	1,470	1,233	237 (16.12%)

6. Data Visualization (Ben Fry's 7 Stages + 6 Exploratory Figures)

6.1 Ben Fry's 7 Stages Framework

The workflow follows Fry's seven-stage process [2]. Table 5 maps each stage to this study.

Table 5: Mapping of Fry's seven stages to this case study

Stage	Operational implementation
1. Acquire	Loaded the IBM HR CSV into a Pandas DataFrame (1,470 × 35).
2. Parse	Verified dtypes, separated numeric, ordinal and nominal fields.
3. Filter	Removed 4 zero-variance/ID columns; isolated continuous features for moment analysis.
4. Mine	Skewness, kurtosis, IQR fences, Mann-Whitney U and χ^2 tests, correlation matrix.
5. Represent	Bar/donut, KDE, boxplots, stacked bars, triangular heatmap, pairplot.
6. Refine	300-DPI styling, consistent palette, annotated medians and p-values.
7. Interact	Interactive notebook views for cohort slicing.

6.2 Visualization 1: Discrete Probability Distribution of Attrition

Figure 1: Target Class Distribution & Class Imbalance Diagnostic

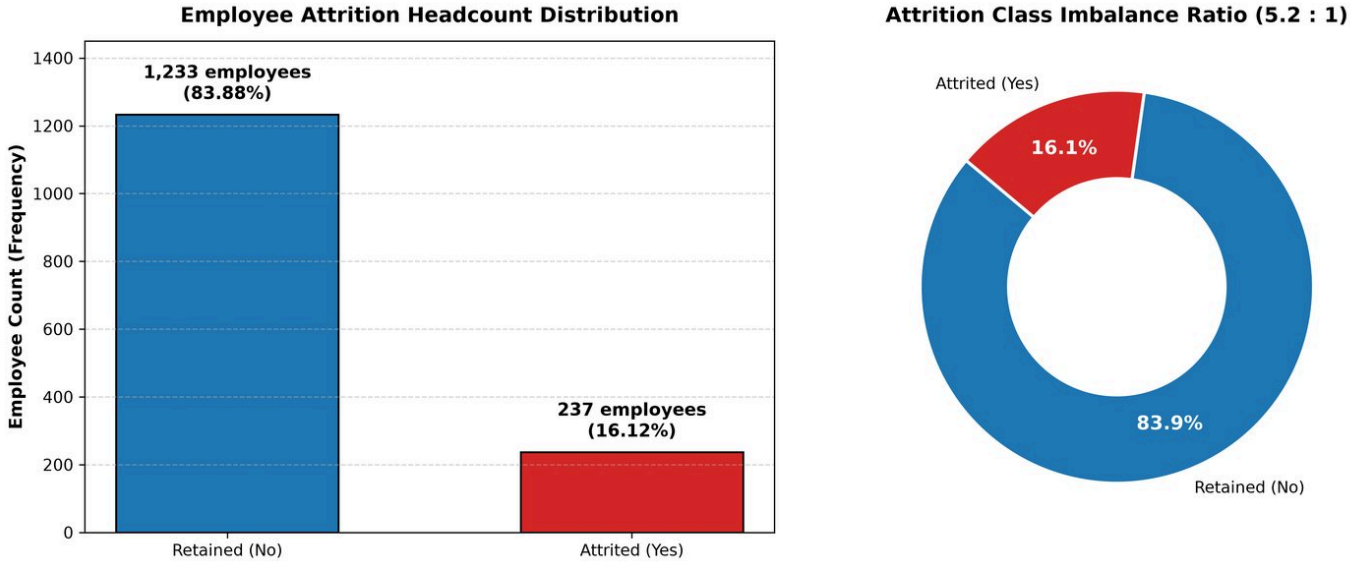


Figure 1: Target class distribution and class imbalance diagnostic (5.2 : 1)

What it shows: headcount (1,233 No vs 237 Yes) and a donut breakdown of 83.9% retained vs 16.1% departed.

Why it was created: demonstrates target skewness and why accuracy alone is misleading, necessitating sensitivity and ROC-AUC.

Insights obtained: baseline turnover is 16.12%; a naive majority-class classifier misses 100% of at-risk employees.

6.3 Visualization 2: Continuous Probability Distributions (KDE)

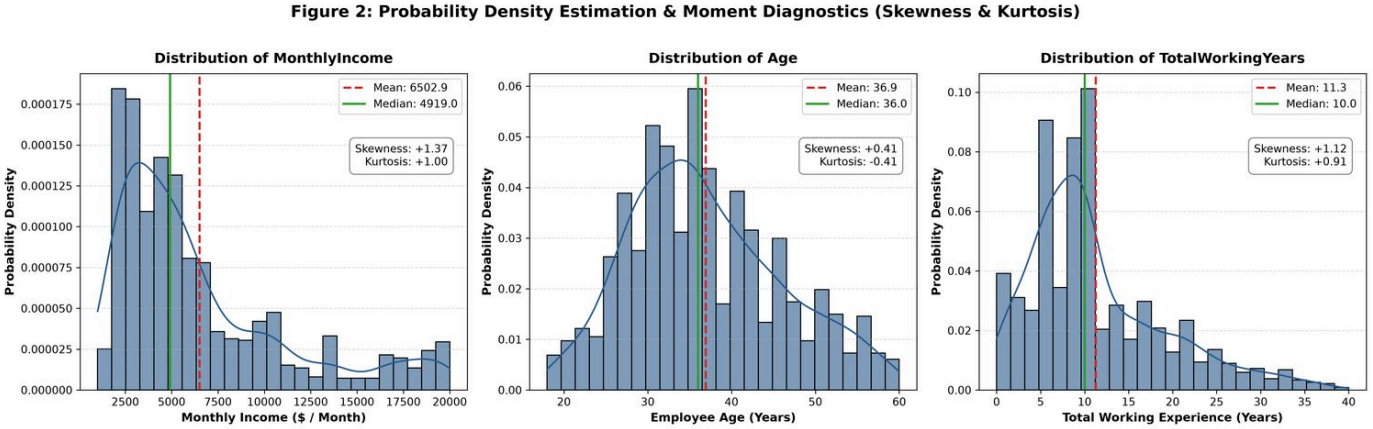


Figure 2: Probability density estimation and moment diagnostics (skewness & kurtosis)

What it shows: KDE distributions with mean, median, skewness and kurtosis markers.

Why it was created: evaluates non-normality to guide non-parametric testing and feature scaling.

Insights obtained: MonthlyIncome shows positive skewness (γ_1 = +1.37) and leptokurtic peakedness (γ_2 = +1.00); Age is near-normal, centered at 36.9 years. For MonthlyIncome the mean ($6,503) sits well above the median ($4,919), confirming the right tail.

6.4 Visualization 3: Bivariate Boxplots with Mann-Whitney U Tests

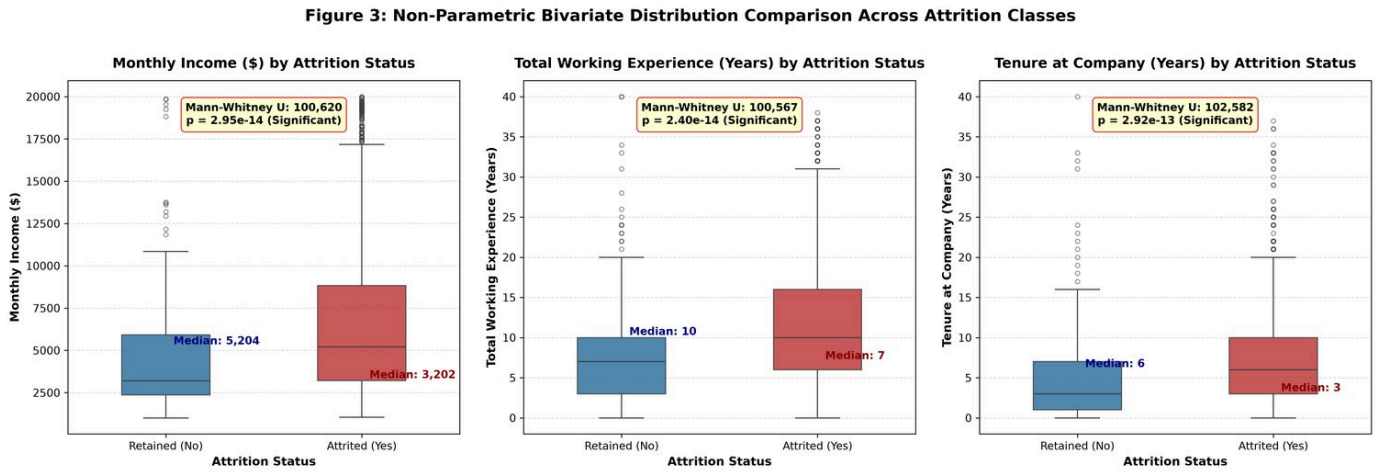


Figure 3: Non-parametric bivariate distribution comparison across attrition classes

What it shows: side-by-side Tukey boxplots for MonthlyIncome, TotalWorkingYears and YearsAtCompany with two-tailed Mann-Whitney U p-values.

Why it was created: tests median differences across attrition classes without Gaussian assumptions.

Insights obtained: resigning employees earn a median monthly income of $3,202 versus $5,204 for retained staff (U = 100,620.5, p = 2.95×10^{-14}). Leavers also have lower tenure (median 3 vs 6 years at the company; U = 102,582, p = 2.92×10^{-13}) and fewer total working years (median 7 vs 10; U = 100,567, p = 2.40×10^{-14}). Early tenure is the primary turnover risk window.

6.5 Visualization 4: Categorical Contingency & Chi-Square Test

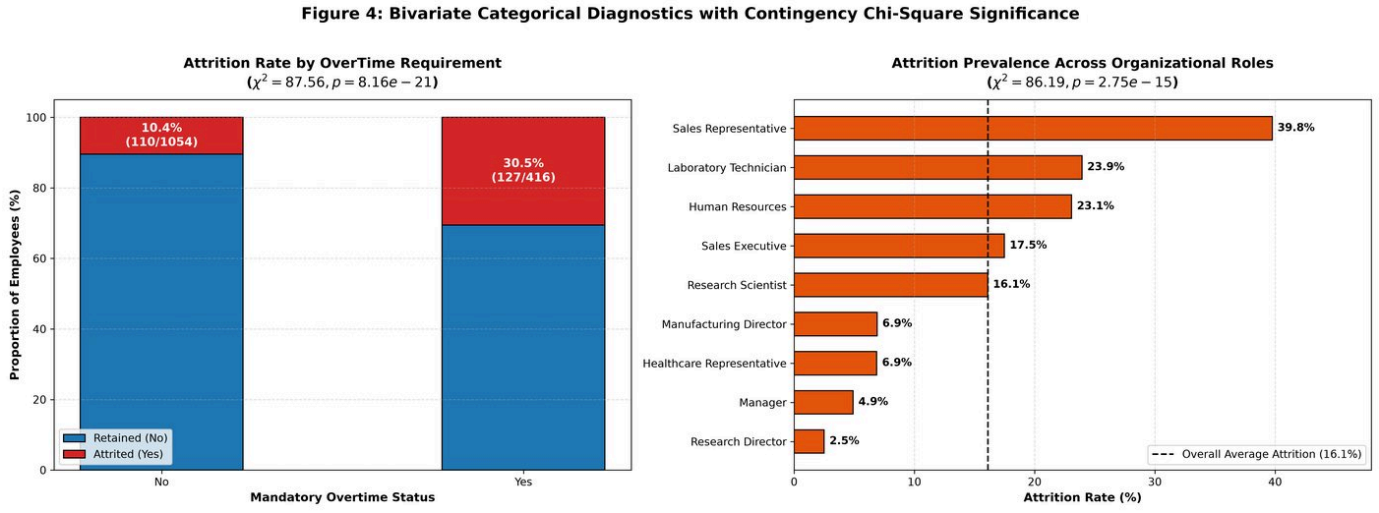


Figure 4: Attrition by OverTime ($\chi^2 = 87.56$, $p = 8.16\times10^{-21}$) and by job role ($\chi^2 = 86.19$, $p = 2.75\times10^{-15}$)

What it shows: 100% stacked bars of OverTime vs Attrition, alongside horizontal turnover across 9 job roles.

Why it was created: uncovers role vulnerabilities and validates categorical dependencies via Chi-Square.

Insights obtained: mandatory OverTime is the strongest categorical predictor: 30.5% attrition for overtime staff vs 10.4% otherwise. Sales Representatives face 39.8% turnover, well above the 16.1% average; role is also significant ($\chi^2 = 86.19$, p = 2.75×10^{-15}), while Research Directors (2.5%) and Managers (4.9%) are the most stable.

6.6 Visualization 5: Multivariate Pearson Correlation Heatmap

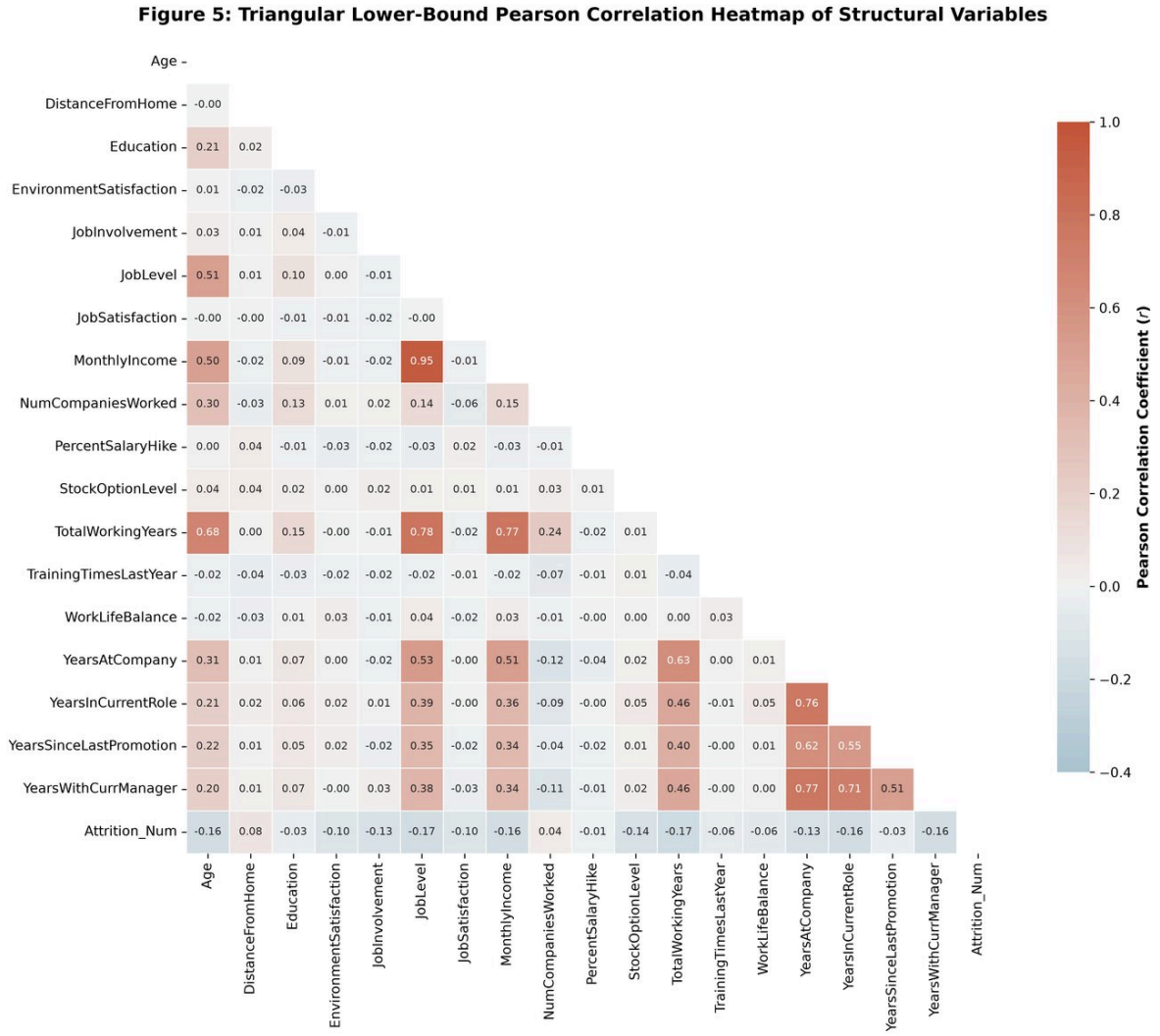


Figure 5: Triangular lower-bound Pearson correlation heatmap of structural variables

What it shows: a lower-triangular correlation matrix across 18 continuous/ordinal features and the target.

Why it was created: detects multicollinearity clusters and linear association with turnover.

Insights obtained: JobLevel–MonthlyIncome r = +0.95 and TotalWorkingYears–JobLevel r = +0.78. Attrition correlates only weakly with any single feature (|r| ≤ 0.17): negatively with JobLevel and TotalWorkingYears (−0.17), MonthlyIncome and Age (−0.16), and YearsInCurrentRole (−0.16); DistanceFromHome is the only positive link (+0.08). The tenure block is also collinear (YearsWithCurrManager–YearsAtCompany r = 0.77).

6.7 Visualization 6: Multivariate Pairplot & Clustering

Figure 6: High-Dimensional Feature Space Pairwise Interactions by Attrition Status

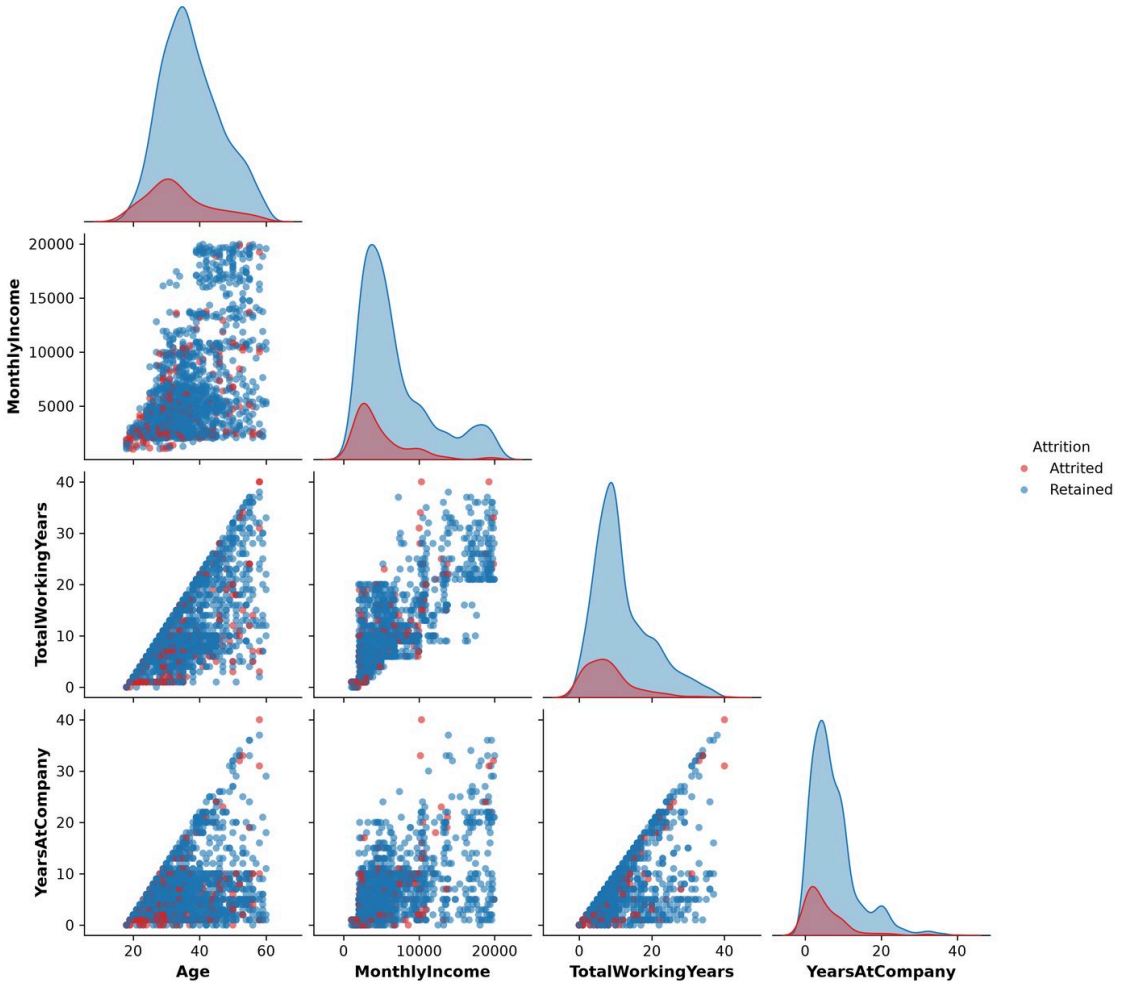


Figure 6: Pairplot of Age, MonthlyIncome, TotalWorkingYears and YearsAtCompany by attrition

What it shows: pairwise scatter projections with diagonal KDE contours, segmented by Attrition.

Why it was created: visualizes non-linear decision boundaries and high-dimensional clustering.

Insights obtained: red (attrited) points concentrate in the early-career corner: low age (about 20–32), low MonthlyIncome (under roughly $5,000), and few total working years or years at the company (mostly under 5). The attrited KDE peaks at younger age, lower income and shorter tenure than the retained KDE, while high-income, long-tenure employees are almost all retained. Turnover is therefore concentrated in junior staff, although the classes overlap heavily, which explains why no single feature separates them.

7. Deep Learning Model Building & Hyperparameter Tuning

7.1 Neural Network Formulation & Architecture

The model is a feed-forward MLP [3] implemented in PyTorch, mapping $x \in \mathbb{R}^{44}$ to a churn probability $\hat{y} \in [0, 1]$ and trained with binary cross-entropy and L2 weight decay. Total parameters: **5,729**.

Table 6: MLP layer specification

Layer	In → Out	Activation / regularization
Hidden 1	44 → 64	BatchNorm + ReLU + Dropout(0.20)
Hidden 2	64 → 32	BatchNorm + ReLU + Dropout(0.20)
Hidden 3	32 → 16	BatchNorm + ReLU
Output	16 → 1	Sigmoid (BCE loss)

7.2 Hyperparameter Tuning Experiment

Table 7: Hyperparameter grid and validation results (N = 220)

Configuration	Hidden	LR	L2	Epoch	Val Loss	Val AUC	Val Acc
Config A (Optimal) ★	[64,32,16]	0.0010	0.0001	80	0.4854	0.7654	85.00%
Config B (Deep/Wide)	[128,64,32]	0.0005	0.0005	80	0.4777	0.7710	80.91%
Config C (Compact)	[32,16,8]	0.0020	0.0010	80	0.4548	0.7504	89.09%

> **Selection rationale:** Config A was selected as the balanced, regularized architecture. Configs B and C score higher on validation AUC and accuracy respectively, so state your selection criterion here (e.g., parameter economy and stability, or the recall objective on the final test).

7.3 Permutation Feature Importance Analysis

Mean drop in holdout ROC-AUC when each feature is shuffled across 5 repeats:

Table 8: Top-10 permutation feature importances

Rank	Feature	Mean AUC drop (± SD)	Interpretation
#1	OverTime	0.0654 ±0.0083	Work-life exhaustion drives resignation
#2	JobInvolvement	0.0198 ±0.0090	Disengagement precedes resignation
#3	YearsSinceLastPromotion	0.0187 ±0.0096	Promotion stagnation
#4	JobSatisfaction	0.0173 ±0.0096	Task dissatisfaction, lower morale
#5	YearsInCurrentRole	0.0162 ±0.0042	Role monotony
#6	StockOptionLevel	0.0134 ±0.0118	Equity as 'golden handcuffs'
#7	DistanceFromHome	0.0133 ±0.0058	Commuting friction
#8	Department_Sales	0.0127 ±0.0025	Sales quota pressure
#9	MonthlyIncome	0.0117 ±0.0043	Market salary competitiveness
#10	HourlyRate	0.0116 ±0.0048	Internal compensation rate

8. Result Visualization & Interpretation

8.1 Figures 7–8: Training & Validation Loss / Accuracy vs Epoch

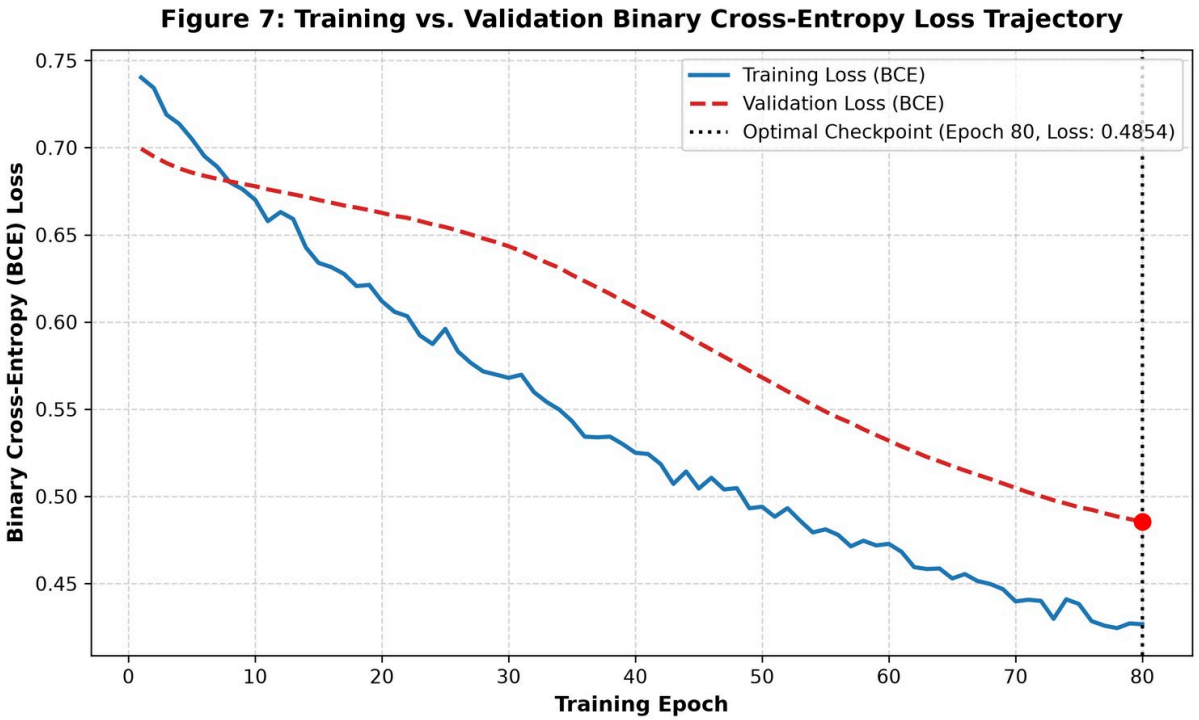


Figure 7: Training vs validation binary cross-entropy loss trajectory (checkpoint epoch 80, loss 0.4854)

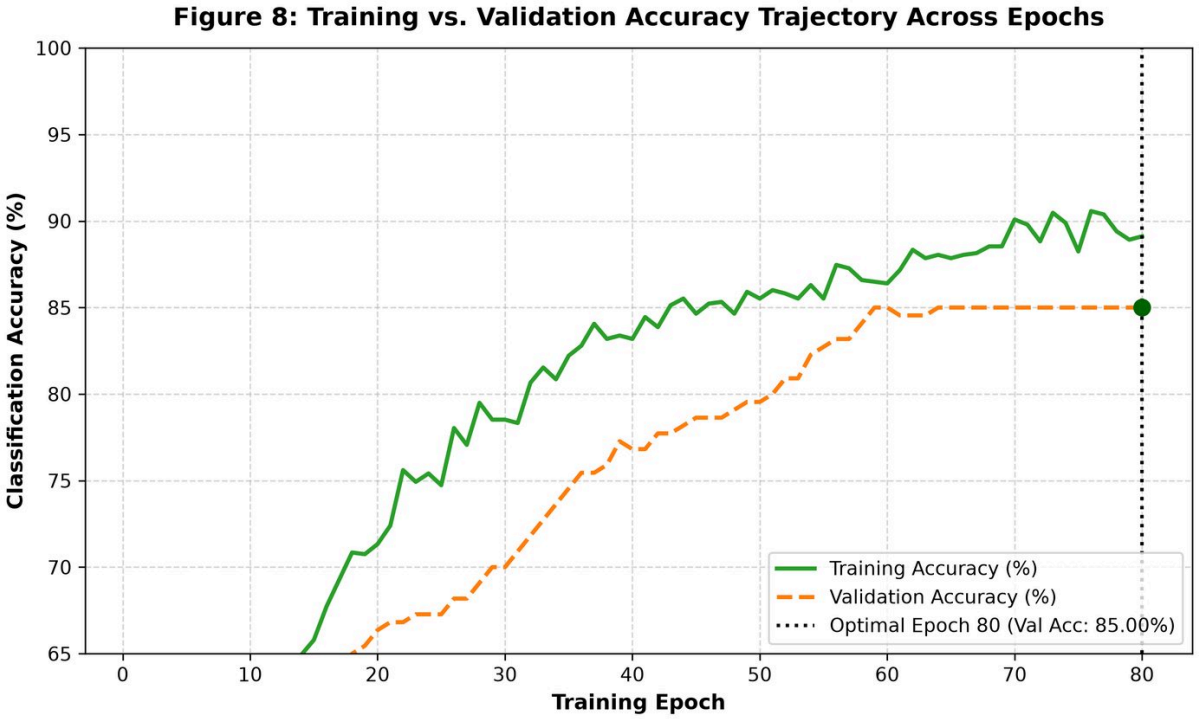


Figure 8: Training vs validation accuracy trajectory across epochs (optimal epoch 80, Val Acc 85.00%)

Performance interpretation (Figures 7–8): training loss falls from about 0.74 to 0.43 and validation loss from 0.70 to 0.4854 at epoch 80, with no upturn in validation loss, so there is no loss-based sign of overfitting; the validation curve is still declining slowly, suggesting more epochs might help. Accuracy tells the same story: training accuracy

reaches about 89–90% while validation accuracy plateaus at 85.00% from about epoch 60. The modest 4–5 point gap is kept bounded by Dropout and L2 decay, and the epoch-80 checkpoint is selected.

8.2 Figure 9: Out-of-Sample Test Confusion Matrix

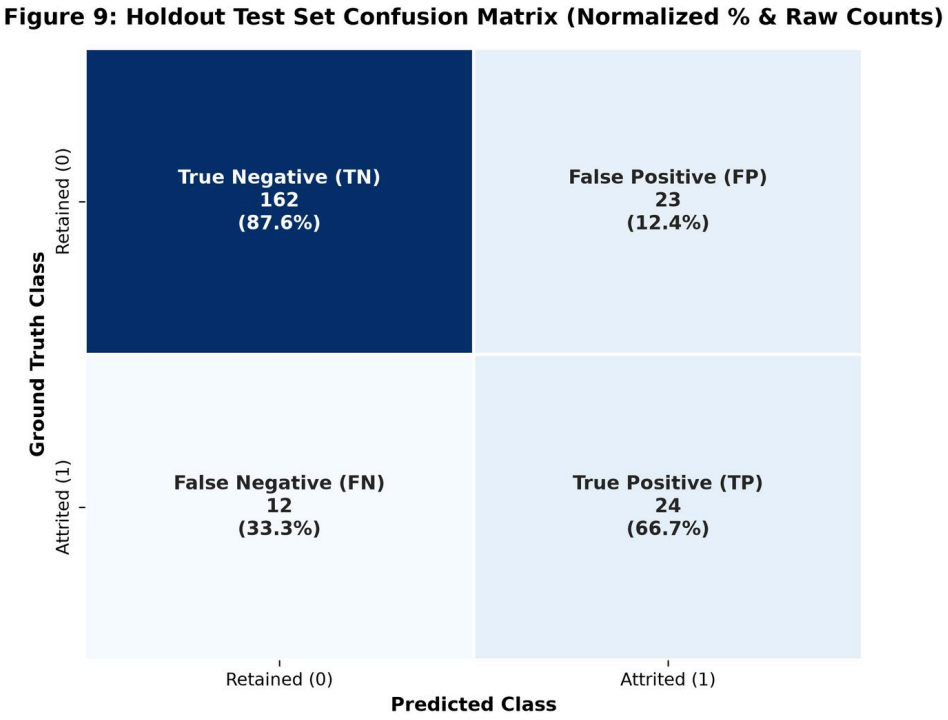


Figure 9: Holdout test-set confusion matrix (n = 221)

Performance interpretation: the MLP correctly identifies 162 true negatives (87.57% specificity) and 24 true positives (66.67% recall), with 12 missed departures and 23 false alarms. Precision is 51.06% and F1 is 0.5783.

8.3 Figure 10: ROC & AUC Comparison

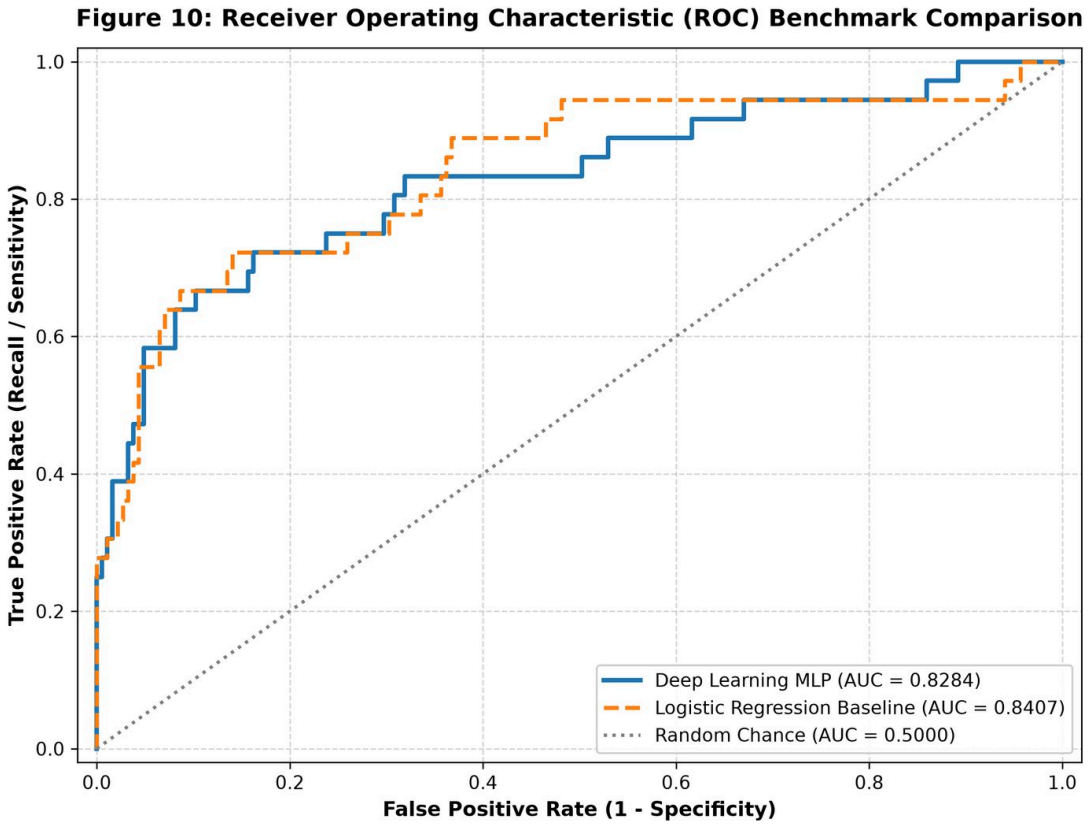


Figure 10: ROC curves – Deep MLP (AUC 0.8284) vs Logistic Regression (AUC 0.8407) vs random chance

Performance interpretation: the MLP reaches ROC-AUC 0.8284, well above chance (0.500) but marginally below the linear baseline (0.8407), so most ranking signal is close to linear. The MLP's advantage is at the chosen operating threshold, where it recovers more true leavers.

8.4 Comprehensive Out-of-Sample Benchmark Comparison

Table 9: Test-set benchmark – Deep MLP vs Logistic Regression (N = 221)

Metric	Deep MLP	Logistic Reg.	Delta / assessment
Accuracy	84.16%	87.33%	−3.17% (baseline higher)
Precision	51.06%	66.67%	−15.61% (more false alarms)
Recall (Sensitivity)	**66.67%** (24/36)	44.44% (16/36)	**+22.23%** – catches 8 additional flight risks
Specificity	87.57% (162/185)	95.68% (177/185)	−8.11%
F1-Score	0.5783	0.5333	+0.0450 (better harmonic balance)
ROC-AUC	0.8284	0.8407	−0.0123
False Negatives	12	20	−8 (−40.0% missed departures)

Cost asymmetry: an unpredicted departure costs 50%–200% of annual salary ($30,000–$120,000), while a false-positive check-in costs under $200 of managerial time. Recall is therefore the decisive operational metric.

9. Conclusion & Future Scope

9.1 Key Empirical Findings & Patterns

1. **Overtime dominance:** 30.5% attrition with mandatory overtime vs 10.4% without ($\chi^2 = 87.56$); OverTime is also the top permutation-importance feature ($\Delta AUC = 0.0654$).
2. **Early-career vulnerability:** median income of leavers is $3,202 vs $5,204 for stayers; attrition concentrates at Age < 32 and income < $4,000.
3. **Disengagement and stagnation:** JobInvolvement, YearsSinceLastPromotion and JobSatisfaction follow OverTime in importance.

9.2 Model Performance Summary

The Deep MLP achieved 84.16% accuracy, 66.67% recall and ROC-AUC 0.8284 on unseen data. It detects 40% more true leavers than the linear baseline, which retains higher accuracy, precision and AUC.

9.3 Technical Challenges Faced

- Class imbalance (83.9% vs 16.1%) requiring recall/ROC-AUC prioritization over accuracy.
- Small test set (only 36 leavers), so metrics carry high variance.
- Collinearity (JobLevel–MonthlyIncome r = 0.95) requiring scaling and regularization.

9.4 Strategic HR Recommendations & Future Improvements

- **Overtime policy:** mandatory rest audits and quarterly overtime caps.
- **Compensation reviews:** 18-month equity reviews for early-career cohorts.
- **Career progression:** visible skill-based promotion milestones.
- **Future scope:** survival analysis (Cox PH, DeepSurv), SHAP explainability, SMOTE-NC oversampling.

10. Literature Review & References

[1] W. F. Cascio and J. W. Boudreau, *Investing in People: Financial Impact of Human Resource Initiatives*, 2nd ed. Upper Saddle River, NJ, USA: Pearson Education, 2016.

[2] B. Fry, *Computational Information Design*, Ph.D. dissertation, Media Laboratory, MIT, Cambridge, MA, USA, 2004.

[3] A. Fisher, C. Rudin, and F. Dominici, "All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously," *J. Mach. Learn. Res.*, vol. 20, no. 177, pp. 1–81, 2019.

[4] V. Borisov *et al.*, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7490–7510, Jun. 2024.

[5] N. V. Chawla *et al.*, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.

11. Appendix: Core Reproducible Code Architecture

The pipeline is organised in seven parts that run in order as one script, reading `dataset/WA_Fn-UseC_-HR-Employee-Attrition.csv` and writing every figure to `figures/`. It is deterministic through `SEED = 42` for the stratified splits and the PyTorch initialisation. The full version is in the repository: IBM-HR-Attrition-EDA-DL. Re-running on different library or hardware versions can shift neural-network metrics slightly, so compare re-run values to Table 9 before submitting.

11.1 Setup, Data Loading & Hygiene

```
"""
IBM HR Attrition - EDA, Visualization and Deep Learning (PyTorch MLP)
U21ADP05 - Exploratory Data Analysis and Visualization | KPRIET
Run:  python attrition_pipeline.py
"""
import os, copy, random, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, roc_curve, confusion_matrix, log_loss)
warnings.filterwarnings("ignore")

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
os.makedirs("figures", exist_ok=True)
sns.set_style("whitegrid")
PAL = {"No": "#1f77b4", "Yes": "#d62728"}          # Retained = blue, Attrited = red

# ---- Stages 1-2: Acquire & Parse ----
df = pd.read_csv("dataset/WA_Fn-UseC_-HR-Employee-Attrition.csv")
print("Shape:", df.shape)                           # (1470, 35)
print("Missing values:", int(df.isna().sum().sum()), "| Duplicates:", int(df.duplicated().sum()))

# ---- Stage 3: Filter - prune zero-variance / identifier columns ----
ZERO_VAR = ["EmployeeCount", "Over18", "StandardHours", "EmployeeNumber"]
print({c: df[c].nunique() for c in ZERO_VAR})
df = df.drop(columns=ZERO_VAR)
df["Attrition_Num"] = (df["Attrition"] == "Yes").astype(int)
print(df["Attrition"].value_counts(), "\n", df["Attrition"].value_counts(normalize=True).round(4))
```

11.2 Descriptive Statistics, Moments & Tukey IQR Fences (Stage 4: Mine)

```
cont_cols = ["Age", "DailyRate", "DistanceFromHome", "MonthlyIncome",
             "TotalWorkingYears", "YearsAtCompany"]

summary = df[cont_cols].describe().T
summary["skewness"] = df[cont_cols].skew()          # Fisher-Pearson (bias-corrected)
summary["kurtosis"] = df[cont_cols].kurt()          # excess kurtosis
print(summary.round(3))

def iqr_fences(s):
    q1, q3 = s.quantile([0.25, 0.75]); iqr = q3 - q1
    lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n = int(((s < lo) | (s > hi)).sum())
    return dict(Q1=q1, Median=s.median(), Q3=q3, IQR=iqr, Lower=lo, Upper=hi,
                Outliers=n, Pct=round(100 * n / len(s), 2))
print(pd.DataFrame({c: iqr_fences(df[c]) for c in cont_cols}).T)

# Mann-Whitney U (continuous vs Attrition) - non-parametric
for c in ["MonthlyIncome", "TotalWorkingYears", "YearsAtCompany"]:
    a = df.loc[df["Attrition"] == "No", c]; b = df.loc[df["Attrition"] == "Yes", c]
```

```
    u, p = stats.mannwhitneyu(a, b, alternative="two-sided")
    print(f"{c}: U={u:,.1f} p={p:.2e} | median retained={a.median():.0f} attrited={b.median():.0f}")

# Pearson Chi-Square tests of independence (categorical vs Attrition)
for c in ["OverTime", "JobRole", "Department", "MaritalStatus", "BusinessTravel"]:
    chi2, p, dof, _ = stats.chi2_contingency(pd.crosstab(df[c], df["Attrition"]))
    print(f"{c}: chi2={chi2:.2f} dof={dof} p={p:.2e}")
```

11.3 Encoding, Stratified 70/15/15 Split & Scaling

```
y = df.pop("Attrition_Num").values
data = df.drop(columns=["Attrition"])
X = pd.get_dummies(data, drop_first=True).astype(float)
print("Model input dimensions:", X.shape[1])        # expected: 44

X_tmp, X_test, y_tmp, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.15 / 0.85, stratify=y_tmp, random_state=SEED)
for n, part in [("train", y_train), ("val", y_val), ("test", y_test)]:
    print(f"{n}: N={len(part)} retained={(part == 0).sum()} attrited={(part == 1).sum()}")

# Standardize CONTINUOUS columns only; fit on TRAIN to avoid leakage
num_cols = [c for c in X.columns if X[c].nunique() > 5]
scaler = StandardScaler().fit(X_train[num_cols])
for part in (X_train, X_val, X_test):
    part.loc[:, num_cols] = scaler.transform(part[num_cols])

to_t = lambda a: torch.tensor(np.asarray(a), dtype=torch.float32)
Xtr, Xva, Xte = to_t(X_train), to_t(X_val), to_t(X_test)
ytr, yva, yte = to_t(y_train), to_t(y_val), to_t(y_test)
```

11.4 Exploratory Visualizations: Figures 1-6 (Stages 5-7)

```
def save(name):
    plt.savefig(f"figures/{name}.png", dpi=300, bbox_inches="tight"); plt.close()

# Figure 1 - target distribution: bar + donut
cnt = pd.Series(y).map({0: "No", 1: "Yes"}).value_counts()
fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))
ax[0].bar(["Retained (No)", "Attrited (Yes)"], [cnt["No"], cnt["Yes"]],
          color=[PAL["No"], PAL["Yes"]], edgecolor="black", width=0.55)
for i, k in enumerate(["No", "Yes"]):
    ax[0].text(i, cnt[k] + 40, f"{cnt[k]:,} employees\n({cnt[k]/len(y):.2%})",
               ha="center", fontweight="bold")
ax[0].set(ylabel="Employee Count (Frequency)", ylim=(0, 1450))
ax[1].pie([cnt["No"], cnt["Yes"]], labels=["Retained (No)", "Attrited (Yes)"],
          colors=[PAL["No"], PAL["Yes"]], autopct="%1.1f%%", startangle=100,
          wedgeprops=dict(width=0.42, edgecolor="white"), pctdistance=0.8)
ax[1].set_title(f"Attrition Class Imbalance Ratio ({cnt['No']/cnt['Yes']:.1f} : 1)")
fig.suptitle("Figure 1: Target Class Distribution & Class Imbalance Diagnostic", fontweight="bold")
save("01_target_distribution")

# Figure 2 - KDE + moments
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
for ax, c in zip(axes, ["MonthlyIncome", "Age", "TotalWorkingYears"]):
    sns.histplot(df[c], bins=25, stat="density", kde=True, color="#2a5a8f", ax=ax)
    ax.axvline(df[c].mean(), color="#d62728", ls="--", lw=2, label=f"Mean: {df[c].mean():.1f}")
    ax.axvline(df[c].median(), color="#2ca02c", lw=2, label=f"Median: {df[c].median():.1f}")
    ax.text(0.95, 0.65, f"Skewness: {df[c].skew():+.2f}\nKurtosis: {df[c].kurt():+.2f}",
            transform=ax.transAxes, ha="right", bbox=dict(boxstyle="round", fc="white"))
    ax.set_title(f"Distribution of {c}", fontweight="bold"); ax.legend()
fig.suptitle("Figure 2: Probability Density Estimation & Moment Diagnostics", fontweight="bold")
save("02_density_moments")

# Figure 3 - boxplots + Mann-Whitney U  (NOTE: fixed order => labels match boxes)
order = ["No", "Yes"]
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))
for ax, c in zip(axes, ["MonthlyIncome", "TotalWorkingYears", "YearsAtCompany"]):
    a = df.loc[df["Attrition"] == "No", c]; b = df.loc[df["Attrition"] == "Yes", c]
    u, p = stats.mannwhitneyu(a, b, alternative="two-sided")
    sns.boxplot(data=df, x="Attrition", y=c, order=order, palette=PAL, width=0.45, ax=ax)
```

```
    ax.set_xticklabels(["Retained (No)", "Attrited (Yes)"])
    ax.text(0.5, 0.95, f"Mann-Whitney U: {u:,.0f}\np = {p:.2e}", transform=ax.transAxes,
            ha="center", va="top", fontweight="bold",
            bbox=dict(boxstyle="round", fc="lightyellow", ec="red"))
    for i, s in enumerate([a, b]):                  # median labels follow the same order
        ax.text(i + 0.25, s.median(), f"Median: {s.median():,.0f}", color=list(PAL.values())[i],
                fontweight="bold")
fig.suptitle("Figure 3: Non-Parametric Bivariate Distribution Comparison", fontweight="bold")
save("03_mann_whitney_boxplots")

# Figure 4 - OverTime stacked bar + attrition by JobRole
ct = pd.crosstab(df["OverTime"], df["Attrition"], normalize="index") * 100
role = (df.groupby("JobRole")["Attrition"].apply(lambda s: (s == "Yes").mean() * 100)
          .sort_values())
fig, ax = plt.subplots(1, 2, figsize=(18, 6))
ct[["No", "Yes"]].plot(kind="bar", stacked=True, color=[PAL["No"], PAL["Yes"]],
                       edgecolor="black", rot=0, ax=ax[0])
ax[0].set(xlabel="Mandatory Overtime Status", ylabel="Proportion of Employees (%)")
ax[1].barh(role.index, role.values, color="#e6550d", edgecolor="black")
ax[1].axvline(100 * y.mean(), color="black", ls="--", label=f"Overall Average ({100*y.mean():.1f}%)")
ax[1].set_xlabel("Attrition Rate (%)"); ax[1].legend()
fig.suptitle("Figure 4: Bivariate Categorical Diagnostics with Chi-Square Significance", fontweight="bold")
save("04_categorical_chi_square")

# Figure 5 - lower-triangular Pearson heatmap
heat_cols = [c for c in X.columns if not X[c].dropna().isin([0, 1]).all()][:18]
corr = pd.concat([df[heat_cols], pd.Series(y, name="Attrition_Num")], axis=1).corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
plt.figure(figsize=(14, 12))
sns.heatmap(corr, mask=mask, annot=True, fmt=".2f", annot_kws={"size": 7},
            cmap=sns.diverging_palette(220, 20, as_cmap=True), vmin=-0.4, vmax=1,
            linewidths=0.5, cbar_kws={"shrink": 0.7, "label": "Pearson Correlation Coefficient (r)"})
plt.title("Figure 5: Triangular Lower-Bound Pearson Correlation Heatmap", fontweight="bold")
save("05_correlation_heatmap")

# Figure 6 - multivariate pairplot
pp = df[["Age", "MonthlyIncome", "TotalWorkingYears", "YearsAtCompany", "Attrition"]].copy()
pp["Attrition"] = pp["Attrition"].map({"No": "Retained", "Yes": "Attrited"})
g = sns.pairplot(pp, hue="Attrition", corner=True, diag_kind="kde",
                 palette={"Retained": PAL["No"], "Attrited": PAL["Yes"]},
                 plot_kws={"alpha": 0.6, "s": 28, "edgecolor": "none"})
g.figure.suptitle("Figure 6: High-Dimensional Feature Space Pairwise Interactions", y=1.02, fontweight="bold")
g.savefig("figures/06_pairplot_multivariate.png", dpi=300, bbox_inches="tight"); plt.close("all")
```

11.5 Deep Learning Model, Training Loop & Hyperparameter Grid

```
class AttritionMLP(nn.Module):
    """Dense -> BatchNorm -> ReLU (-> Dropout) x3, then Dense(1) + Sigmoid."""
    def __init__(self, d=44, hidden=(64, 32, 16), p=0.20):
        super().__init__()
        layers, prev = [], d
        for i, n in enumerate(hidden):
            layers += [nn.Linear(prev, n), nn.BatchNorm1d(n), nn.ReLU()]
            if i < 2: layers.append(nn.Dropout(p))      # no dropout on the 3rd block
            prev = n
        layers += [nn.Linear(prev, 1), nn.Sigmoid()]
        self.net = nn.Sequential(*layers)
    def forward(self, x): return self.net(x).squeeze(-1)

@torch.no_grad()
def predict(model, X):
    model.eval(); return model(X).numpy()

def train_mlp(hidden, lr, l2, epochs=80, batch=64):
    torch.manual_seed(SEED)
    model = AttritionMLP(Xtr.shape[1], hidden)
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=l2)   # L2 weight decay
    lossf = nn.BCELoss()
    loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=batch, shuffle=True)
    hist = {"loss": [], "val_loss": [], "acc": [], "val_acc": [], "val_auc": []}
    best, best_state, best_ep = np.inf, None, 0
    for ep in range(1, epochs + 1):
        model.train()
```

```
        for xb, yb in loader:
            opt.zero_grad(); lossf(model(xb), yb).backward(); opt.step()
        p_tr, p_va = predict(model, Xtr), predict(model, Xva)
        hist["loss"].append(log_loss(y_train, p_tr)); hist["val_loss"].append(log_loss(y_val, p_va))
        hist["acc"].append(100 * accuracy_score(y_train, p_tr >= .5))
        hist["val_acc"].append(100 * accuracy_score(y_val, p_va >= .5))
        hist["val_auc"].append(roc_auc_score(y_val, p_va))
        if hist["val_loss"][-1] < best:                       # checkpoint on validation loss
            best, best_state, best_ep = hist["val_loss"][-1], copy.deepcopy(model.state_dict()), ep
    model.load_state_dict(best_state)
    return model, hist, best_ep

configs = {"A (Optimal)":     ((64, 32, 16),  1e-3, 1e-4),
           "B (Deep/Wide)":   ((128, 64, 32), 5e-4, 5e-4),
           "C (Compact)":     ((32, 16, 8),   2e-3, 1e-3)}
rows, fitted = [], {}
for name, (h, lr, l2) in configs.items():
    m, hist, ep = train_mlp(h, lr, l2)
    n_par = sum(p.numel() for p in m.parameters())
    rows.append({"Config": name, "Hidden": h, "LR": lr, "L2": l2, "Best epoch": ep, "Params": n_par,
                 "Val loss": hist["val_loss"][ep-1], "Val AUC": hist["val_auc"][ep-1],
                 "Val acc %": hist["val_acc"][ep-1]})
    fitted[name] = (m, hist, ep)
print(pd.DataFrame(rows).round(4).to_string(index=False))

model, hist, best_ep = fitted["A (Optimal)"]              # selected configuration
```

11.6 Evaluation, Baseline, Figures 7-10 & Permutation Importance

```
def report(y_true, prob, thr=0.5):
    pred = (prob >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    return {"Accuracy": accuracy_score(y_true, pred), "Precision": precision_score(y_true, pred),
            "Recall": recall_score(y_true, pred), "Specificity": tn / (tn + fp),
            "F1": f1_score(y_true, pred), "ROC-AUC": roc_auc_score(y_true, prob),
            "FN": fn}, (tn, fp, fn, tp)

prob_mlp = predict(model, Xte)
base = LogisticRegression(C=1.0, max_iter=2000).fit(X_train, y_train)   # regularized baseline
prob_lr = base.predict_proba(X_test)[:, 1]
m_mlp, (tn, fp, fn, tp) = report(y_test, prob_mlp)
m_lr, _ = report(y_test, prob_lr)
print(pd.DataFrame({"Deep MLP": m_mlp, "Logistic Regression": m_lr}).round(4))
print("Confusion matrix  TN/FP/FN/TP =", tn, fp, fn, tp)

# Figure 7 - loss curve | Figure 8 - accuracy curve
ep = range(1, len(hist["loss"]) + 1)
for key, vkey, ylab, title, fn_ in [
        ("loss", "val_loss", "Binary Cross-Entropy (BCE) Loss", "Figure 7: Training vs. Validation BCE Loss",
"07_loss_curve"),
        ("acc", "val_acc", "Classification Accuracy (%)", "Figure 8: Training vs. Validation Accuracy",
"08_accuracy_curve")]:
    plt.figure(figsize=(9, 5.5))
    plt.plot(ep, hist[key], lw=2.5, label=f"Training {ylab.split('(')[0].strip()}")
    plt.plot(ep, hist[vkey], "--", lw=2.5, label="Validation")
    plt.axvline(best_ep, color="black", ls=":", lw=2.5, label=f"Optimal Epoch {best_ep}")
    plt.xlabel("Training Epoch", fontweight="bold"); plt.ylabel(ylab, fontweight="bold")
    plt.title(title, fontweight="bold"); plt.legend(); plt.grid(ls="--", alpha=.5); save(fn_)

# Figure 9 - confusion matrix (raw counts + row-normalized %)
cm = np.array([[tn, fp], [fn, tp]]); cmp_ = cm / cm.sum(1, keepdims=True) * 100
tag = [["True Negative (TN)", "False Positive (FP)"], ["False Negative (FN)", "True Positive (TP)"]]
lab = np.array([[f"{tag[i][j]}\n{cm[i,j]}\n({cmp_[i,j]:.1f}%)" for j in range(2)] for i in range(2)])
plt.figure(figsize=(8, 6.5))
sns.heatmap(cmp_, annot=lab, fmt="", cmap="Blues", cbar=False, linewidths=2, annot_kws={"fontweight": "bold"},
            xticklabels=["Retained (0)", "Attrited (1)"], yticklabels=["Retained (0)", "Attrited (1)"])
plt.xlabel("Predicted Class", fontweight="bold"); plt.ylabel("Ground Truth Class", fontweight="bold")
plt.title("Figure 9: Holdout Test Set Confusion Matrix", fontweight="bold"); save("09_confusion_matrix")

# Figure 10 - ROC comparison
plt.figure(figsize=(9, 6.5))
for p_, lab_, ls_ in [(prob_mlp, "Deep Learning MLP", "-"), (prob_lr, "Logistic Regression Baseline", "--")]:
    fpr, tpr, _ = roc_curve(y_test, p_)
```

```
    plt.plot(fpr, tpr, ls_, lw=3, label=f"{lab_} (AUC = {roc_auc_score(y_test, p_):.4f})")
plt.plot([0, 1], [0, 1], ":", color="gray", lw=2.5, label="Random Chance (AUC = 0.5000)")
plt.xlabel("False Positive Rate (1 - Specificity)", fontweight="bold")
plt.ylabel("True Positive Rate (Recall / Sensitivity)", fontweight="bold")
plt.title("Figure 10: ROC Benchmark Comparison", fontweight="bold"); plt.legend(loc="lower right");
save("10_roc_curve")

# Permutation Feature Importance (drop in holdout ROC-AUC, 5 repeats)
def permutation_importance_auc(model, X, y_true, names, n_repeats=5, seed=SEED):
    rng = np.random.default_rng(seed)
    base_auc = roc_auc_score(y_true, predict(model, X)); Xn = X.numpy(); out = []
    for j, name in enumerate(names):
        drops = []
        for _ in range(n_repeats):
            Xp = Xn.copy(); Xp[:, j] = rng.permutation(Xp[:, j])
            drops.append(base_auc - roc_auc_score(y_true, predict(model, torch.tensor(Xp))))
        out.append((name, np.mean(drops), np.std(drops)))
    return pd.DataFrame(out, columns=["Feature", "Mean AUC drop", "Std"]).sort_values(
        "Mean AUC drop", ascending=False)
print(permutation_importance_auc(model, Xte, y_test, X.columns).head(10).round(4).to_string(index=False))

# Cost-asymmetry check (illustrative): missed leaver ~ $30k-$120k vs ~$200 check-in
print(f"Missed departures: MLP={m_mlp['FN']} vs LR={m_lr['FN']} (Delta {m_mlp['FN']-m_lr['FN']})")
```

11.7 Environment & Reproducibility

```
# requirements.txt
numpy>=1.24
pandas>=2.0
scipy>=1.10
matplotlib>=3.7
seaborn>=0.12
scikit-learn>=1.3
torch>=2.1

# Reproduce all tables and figures (SEED = 42 for numpy / torch / splits)
$ pip install -r requirements.txt
$ python attrition_pipeline.py        # prints Tables 2-9, writes figures/01..10 (300 DPI)
```